

CSE 2201: Database Management Systems

Lecture 13: Normalization I – 1NF, 2NF and 3NF with Examples

Md Mynoddin

Assistant Professor (CSE), RMSTU

Department of Computer Science and Engineering

Lecture Outcomes

After this lecture, students should be able to:

- 1 explain why normalization is needed in relational design;
- 2 identify unnormalized and 1NF tables;
- 3 detect partial dependencies and explain 2NF;
- 4 detect transitive dependencies and explain 3NF;
- 5 decompose a relation step by step into 1NF, 2NF and 3NF;
- 6 check whether the decomposition is meaningful and lossless;
- 7 translate normalized relations into SQL tables.

Main Theme

Normalization is a systematic way to remove mixed facts from one large table and store each fact in the proper table.

Quick Revision from Lecture 12

Concept	Meaning
Functional dependency	$X \rightarrow Y$ means values of X determine values of Y .
Superkey	K is a superkey if $K \rightarrow R$.
Candidate key	Minimal superkey.
Prime attribute	Attribute that is part of at least one candidate key.
Non-prime attribute	Attribute that is not part of any candidate key.
Lossless decomposition	Original relation can be reconstructed without spurious tuples.

Why Normalize?

Reduce redundancy

Same fact should not be repeated unnecessarily.

Avoid anomalies

Prevent update, insertion and deletion problems.

Improve meaning

Each relation should represent one main entity or relationship.

Important Warning

Normalization improves logical design. It is not mainly about saving disk space; it is about correctness and maintainability.

Running Example: Student-Course Result

Result(sid, sname, dept, cid, title, instructor, office, grade)

sid	sname	dept	cid	title	instructor	office	grade
S01	Asha	CSE	C101	DBMS	Rahman	R201	A
S01	Asha	CSE	C102	OOP	Karim	R205	B+
S02	Bimal	CSE	C101	DBMS	Rahman	R201	A-
S03	Chaya	EEE	E201	Circuits	Akter	E101	B

Likely FDs

$\text{sid} \rightarrow \text{sname}, \text{dept}$

$\text{cid} \rightarrow \text{title}, \text{instructor}, \text{instructor} \rightarrow \text{office}$

$\text{sid}, \text{cid} \rightarrow \text{grade}$

Candidate Key in the Running Example

Relation:

Result(sid, sname, dept, cid, title, instructor, office, grade)

Candidate key

A student receives a grade for a course. Therefore:

$\text{sid, cid} \rightarrow \text{sname, dept, title, instructor, office, grade}$

So sid, cid is a candidate key.

Prime and non-prime attributes

- Prime attributes: sid, cid
- Non-prime attributes: sname, dept, title, instructor, office, grade

Normalization Roadmap

Normal Form	Main Problem Removed	Simple Question
1NF	Repeating groups and non-atomic values	Is every cell atomic?
2NF	Partial dependency on part of a composite key	Does every non-prime attribute depend on the whole key?
3NF	Transitive dependency through non-key attributes	Does a non-key attribute determine another non-key attribute?

Progressive Rule

3NF assumes 2NF, and 2NF assumes 1NF.

First Normal Form: 1NF

Definition

A relation is in **First Normal Form (1NF)** if every attribute value is atomic: each cell contains a single indivisible value.

Violations of 1NF

- multiple phone numbers in one cell;
- comma-separated course list in one attribute;
- repeating columns such as phone1, phone2, phone3;
- nested table stored inside one cell.

1NF Example: Not Atomic

student_id	student_name	phones
2201	Nowshin	01711, 01822
2202	Avijit	01933

Problem

The phones attribute stores multiple values in one cell. This violates 1NF.

1NF Solution

`Student(student_id, student_name)`

`StudentPhone(student_id, phone)`

1NF SQL Implementation

```
CREATE TABLE student (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(80) NOT NULL  
);  
  
CREATE TABLE student_phone (  
    student_id INT NOT NULL,  
    phone VARCHAR(20) NOT NULL,  
    PRIMARY KEY (student_id, phone),  
    FOREIGN KEY (student_id) REFERENCES student(student_id)  
);
```

Design Idea

A multivalued attribute in ER modeling usually becomes a separate relation in relational design.

Second Normal Form: 2NF

Definition

A relation is in **Second Normal Form (2NF)** if:

- it is already in 1NF, and
- every non-prime attribute is fully functionally dependent on the whole candidate key.

Partial Dependency

A partial dependency occurs when a non-prime attribute depends on only part of a composite candidate key.

When 2NF matters most

2NF mainly matters when the candidate key is composite, such as (*student_id*, *course_id*).

2NF Violation in Running Example

Relation:

Result(sid, sname, dept, cid, title, instructor, office, grade)

Candidate key:

sid, cid

Dependency	Explanation	2NF Status
sid $\rightarrow$ sname, dept	student facts depend only on sid	Violation
cid $\rightarrow$ title, instructor	course facts depend only on cid	Violation
sid, cid $\rightarrow$ grade	grade depends on whole key	OK

Decompose to 2NF

From:

Result(sid, sname, dept, cid, title, instructor, office, grade)

Create:

Student(sid, sname, dept)

Course(cid, title, instructor, office)

Enrollment(sid, cid, grade)

What changed?

- Student data depends on sid only.
- Course data depends on cid only.
- Enrollment data depends on the composite key sid, cid.

But 2NF May Still Have a Problem

In the 2NF relation:

`Course(cid, title, instructor, office)`

Suppose:

`cid → title, instructor`

`instructor → office`

Then:

`cid → instructor → office`

Problem

office depends on cid through instructor. This is a transitive dependency.

Third Normal Form: 3NF Intuition

Simple Intuition

A relation is in 3NF if non-key attributes do not determine other non-key attributes.

Transitive Dependency

A transitive dependency occurs when:

$$K \rightarrow A \quad \text{and} \quad A \rightarrow B$$

where K is a key and A, B are non-prime attributes.

Example

`cid → instructor, instructor → office`

So office should be moved to an Instructor relation.

Formal 3NF Condition

A relation schema R is in **3NF** with respect to FD set F if, for every FD:

$$\alpha \rightarrow \beta$$

at least one of the following is true:

- 1 $\alpha \rightarrow \beta$ is trivial;
- 2 α is a superkey for R ;
- 3 every attribute in $\beta - \alpha$ is prime.

Simplified teaching version

For most introductory examples: every non-key fact should depend on the key, the whole key, and nothing but the key.

Decompose to 3NF

From 2NF relation:

`Course(cid, title, instructor, office)`

With:

`cid → title, instructor,    instructor → office`

Create:

`Course(cid, title, instructor)`
`Instructor(instructor, office)`

Now

- Course stores course facts.
- Instructor stores instructor facts.
- Office is stored once per instructor.

Final 3NF Design for Running Example

```
Student(sid, sname, dept)
Course(cid, title, instructor)
Instructor(instructor, office)
Enrollment(sid, cid, grade)
```

Foreign keys

- `Course.instructor` references `Instructor.instructor`
- `Enrollment.sid` references `Student.sid`
- `Enrollment.cid` references `Course.cid`

Result

Redundancy is reduced, and student, course, instructor and enrollment facts are separated.

3NF SQL Schema Example: Entity Tables

```
CREATE TABLE student (  
    sid VARCHAR(10) PRIMARY KEY,  
    sname VARCHAR(80) NOT NULL,  
    dept VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE instructor (  
    instructor VARCHAR(80) PRIMARY KEY,  
    office VARCHAR(20) NOT NULL  
);
```

Meaning

Student facts and instructor facts are stored independently.

3NF SQL Schema Example: Relationship Tables

```
CREATE TABLE course (  
  cid VARCHAR(10) PRIMARY KEY,  
  title VARCHAR(100) NOT NULL,  
  instructor VARCHAR(80) NOT NULL,  
  FOREIGN KEY (instructor) REFERENCES instructor(instructor)  
);
```

```
CREATE TABLE enrollment (  
  sid VARCHAR(10),  
  cid VARCHAR(10),  
  grade VARCHAR(2),  
  PRIMARY KEY (sid, cid),  
  FOREIGN KEY (sid) REFERENCES student(sid),  
  FOREIGN KEY (cid) REFERENCES course(cid)  
);
```

Example 2: Order Table

Unnormalized/mixed relation:

```
OrderLine(order_id, order_date, customer_id, customer_name,  
          product_id, product_name, qty, unit_price)
```

Candidate key:

`order_id, product_id`

Likely FDs:

`order_id → order_date, customer_id`

`customer_id → customer_name`

`product_id → product_name, unit_price`

`order_id, product_id → qty`

Question

Which dependencies violate 2NF and which violate 3NF?

Order Table: Normalized Design

```
Customer(customer_id, customer_name)
Orders(order_id, order_date, customer_id)
Product(product_id, product_name, unit_price)
OrderItem(order_id, product_id, qty)
```

Why this is better

- Customer name is stored once per customer.
- Product name and price are stored once per product.
- Ordered quantity belongs to the order-product relationship.

Example 3: Employee Project Assignment

Relation:

`Assign(emp_id, emp_name, dept_id, dept_name, project_id, project_name, hours)`

Candidate key:

`emp_id, project_id`

FDs:

`emp_id → emp_name, dept_id`

`dept_id → dept_name`

`project_id → project_name`

`emp_id, project_id → hours`

Practice

Decompose this relation into 3NF.

Employee Project: 3NF Answer

```
Employee(emp_id, emp_name, dept_id)
Department(dept_id, dept_name)
Project(project_id, project_name)
Assignment(emp_id, project_id, hours)
```

Reasoning

- Employee facts depend on emp_id.
- Department facts depend on dept_id.
- Project facts depend on project_id.
- Assignment hours depend on both employee and project.

Checklist: How to Normalize in Exams

- 1 Write the original relation schema.
- 2 List given or assumed functional dependencies.
- 3 Identify candidate key(s).
- 4 Check 1NF: are all attribute values atomic?
- 5 Check 2NF: does any non-prime attribute depend on part of a composite key?
- 6 Check 3NF: does any non-prime attribute determine another non-prime attribute?
- 7 Decompose carefully and write primary keys.
- 8 Add foreign keys to preserve relationships.
- 9 State assumptions if the problem is ambiguous.

Common Student Mistakes

Avoid these

- Decomposing without identifying candidate keys first.
- Treating every repeated value as a separate table without understanding FDs.
- Forgetting the relationship table for M:N relationships.
- Removing attributes but not adding foreign keys.
- Confusing 2NF partial dependency with 3NF transitive dependency.
- Thinking normalization means every table must have only two columns.

In-Class Exercises

Normalize each relation up to 3NF.

- 1 Library loan: member_id, member_name, book_id, title, author, issue_date, due_date
- 2 Doctor appointment: patient_id, patient_name, doctor_id, doctor_name, dept, appointment_date, fee
- 3 Hostel allocation: student_id, student_name, hostel_id, hostel_name, room_id, room_type, allocation_date

Submission Style

For each problem: list FDs, candidate key, 1NF/2NF/3NF status, and final decomposed relations.

Lecture Summary

- 1NF removes multivalued or non-atomic cells.
- 2NF removes partial dependency on part of a composite key.
- 3NF removes transitive dependency through non-key attributes.
- Normalization uses FDs and candidate keys, not just visual appearance.
- A good normalized design should keep facts separated and relationships connected by foreign keys.

Next Lecture

We continue with BCNF, multivalued dependencies, 4NF and 5NF.